

Lezione 2 - Architetture dei microservizi

Architettura dei Microservizi

Fino ad ora abbiamo visto i microservizi da un punto di vista concettuale e commerciale — cosa sono, perché convengono, come scalano e come gestiscono i guasti.

Ma non abbiamo ancora risposto a una domanda fondamentale: **come sono strutturati internamente e come comunicano tra loro?**

Un singolo microservizio, preso da solo, assomiglia molto a quello che abbiamo costruito negli esercizi precedenti — una applicazione Spring con Controller, Service, DAO, Entity e DTO. La vera complessità non sta nel singolo microservizio, ma in **come questi microservizi vengono esposti al mondo esterno** e in **come il sistema nel suo insieme viene orchestrato**.

Pensiamo all'esempio di Amazon: il client — che sia un browser, un'app mobile o un'app desktop — non può conoscere gli indirizzi IP di tutti i microservizi interni. Non può sapere dove gira il microservizio degli ordini, dove gira quello dei pagamenti, quante istanze sono attive in un dato momento. Avere questa complessità esposta direttamente al client sarebbe insostenibile.

La soluzione è introdurre componenti infrastrutturali dedicati che si occupano di orchestrare la comunicazione — il più importante dei quali è il Gateway:

- **È lui il punto di ingresso unico tra il mondo esterno e la rete interna dei microservizi, ed è attorno a lui che si costruisce l'intera architettura.**

Le Due Architetture dei Microservizi

Un'applicazione a microservizi si struttura su 2 livelli architetturali distinti — uno interno e uno esterno — **che insieme definiscono sia come ogni microservizio è costruito che come comunica con gli altri**.

Architettura Interna

L'architettura interna riguarda la struttura di ogni singolo microservizio preso individualmente.

Non è nulla di nuovo — è esattamente l'architettura

Controller/Service/Repository che abbiamo costruito negli esercizi precedenti, con Entity, DTO, Mapper e gestione degli errori.

Ogni microservizio è a tutti gli effetti una piccola applicazione Spring autonoma, con il proprio database dedicato.

Se il microservizio prevede anche una [View](#) — ad esempio una pagina HTML — si aggiunge anche il layer di presentazione. Altrimenti, come nel nostro caso, il microservizio espone solo API REST.

Architettura Esterna

L'architettura esterna riguarda invece i **meccanismi di comunicazione** tra i microservizi e con il mondo esterno. Guardando l'immagine, il punto cruciale è che i client — web e mobile — non comunicano mai direttamente con i singoli microservizi. Passano sempre attraverso l'**API Gateway**, che fa da unico punto di ingresso verso la rete interna.

I meccanismi di comunicazione tra i microservizi sono due:

- **Chiamate REST** — comunicazione sincrona tramite [HTTP](#), come abbiamo già visto
- **Code e Broadcast** — [comunicazione asincrona tramite](#) sistemi di messaggistica, come nell'esempio dell'email di conferma dopo un pagamento

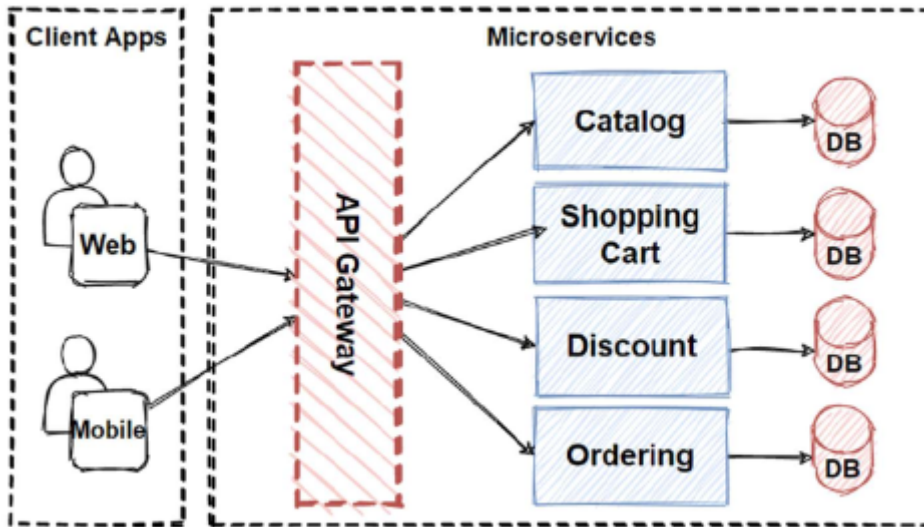
👁 Il confine tra architettura interna ed esterna è netto:

- l'interna riguarda come un microservizio è fatto,
- l'esterna riguarda come i microservizi parlano tra loro.

Sono due preoccupazioni completamente separate — e questa separazione è uno dei principi fondamentali del design a microservizi.

Esempio di struttura

Per comprendere meglio questa suddivisione analizziamo questa immagine:



Abbiamo due client — un'applicazione web e un'applicazione mobile — che vogliono accedere ai servizi di un e-commerce. Entrambi inviano le loro richieste HTTP non direttamente ai microservizi, ma all'**API Gateway**:

- **il punto di ingresso unico dell'applicazione, che fa da "portiere" smistando ogni richiesta al microservizio corretto.**

In base alla richiesta ricevuta, il Gateway la inoltra al microservizio appropriato:

- Catalog per consultare i prodotti,
- Shopping Cart per gestire il carrello,
- Discount per applicare sconti,
- Ordering per gestire gli ordini.

Ognuno di questi microservizi ha il **proprio database dedicato**:

- **nessuno condivide i dati con gli altri, rispettando il principio di autonomia che abbiamo visto in precedenza.**

La DMZ — Zona Demilitarizzata

Nell'architettura a microservizi la sicurezza della rete interna è fondamentale — i **microservizi devono poter comunicare tra loro in modo sicuro, senza essere esposti direttamente a internet.**

È qui che entra in gioco la **DMZ** (Demilitarized Zone).

La **DMZ** è:

- **una zona "franca" che separa la rete non protetta (internet) dalla LAN interna dove vivono i microservizi.**

Funziona come un buffer di sicurezza a doppio strato:

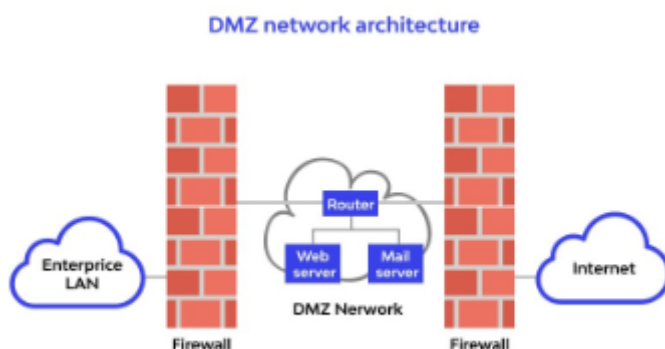
- **Firewall esterno:**
 - primo filtro che blocca le chiamate indesiderate provenienti da internet.
 - La rete esterna conosce solo l'IP del firewall, non sa nulla di quello che c'è dietro.
- **API Gateway:**
 - unico punto di accesso autorizzato alla rete interna.
 - Tutte le richieste legittime passano attraverso di lui.
- **Firewall interno:**
 - secondo filtro di protezione nel caso in cui il firewall esterno venga superato.

Grazie a questa struttura, i microservizi nella rete interna possono comunicare tra loro nella fiducia di un ambiente sicuro — senza preoccuparsi di filtrare o validare le richieste in ingresso, perché ci ha già pensato la DMZ.

≡ Analogia Condominio:

È esattamente come un condominio con cancello elettronico e portiere:

- il cancello (firewall esterno) blocca gli estranei,
- il portiere (Gateway) decide chi può entrare,
- e una volta dentro gli inquilini (microservizi) possono muoversi liberamente in un ambiente protetto.



Il Boundary — Il Perimetro dell'Applicazione

Tutto ciò che abbiamo visto finora — microservizi, database, Gateway, DMZ — vive all'interno di un perimetro logico e infrastrutturale ben definito che

separa l'intero sistema dai sistemi esterni.

Questo confine si chiama **Boundary**.

Il Boundary non contiene solo i microservizi applicativi, ma tutto ciò che è necessario per far funzionare il sistema nel suo insieme:

1. **Microservizi e database** — **i servizi applicativi veri e propri, ognuno con il proprio database dedicato come abbiamo già visto.**
2. **Servizi infrastrutturali** — componenti che NON contengono logica di business ma sono **INDISPENSABILI** per il funzionamento dell'architettura:
 - **Configurazione** — **gestisce le configurazioni centralizzate di tutti i microservizi**
 - **Registry** — **tiene traccia di quali microservizi sono attivi e dove si trovano**
 - **Gateway** — **il punto di ingresso unico che abbiamo già analizzato**
3. **Sistema di messaggistica a code:**
 - **strumenti come Kafka o RabbitMQ che gestiscono la comunicazione asincrona tra i microservizi — l'equivalente infrastrutturale dell'esempio dell'email di conferma che abbiamo visto.**
4. **Orchestrazione — Kubernetes:**
 - **lo strumento che gestisce automaticamente il ciclo di vita dei microservizi:**
 - **li avvia,**
 - **li scala,**
 - **li riavvia in caso di guasto.**
5. **Componenti di sicurezza interna:**
 - **gestione delle identità e degli accessi all'interno della rete interna.**
6. **Logging e monitoring condivisi:**
 - **sistemi centralizzati per monitorare lo stato di tutti i microservizi e raccogliere i log in un unico posto:**
 - **indispensabili in un sistema distribuito dove i problemi possono manifestarsi in qualsiasi punto della rete.**

🕒 **Il Boundary è quindi la risposta alla domanda "dove finisce la nostra applicazione e dove inizia il mondo esterno?".**

Tutto ciò che è dentro il Boundary è sotto il controllo e la responsabilità del team di sviluppo — tutto ciò che è fuori è internet, cioè territorio non

controllato.

Il Gateway

Il **Gateway** è la porta logica dell'applicazione a microservizi — **l'entry point unico che collega il mondo esterno all'ecosistema dei microservizi interni al Boundary.**

È il componente più strategico dell'architettura esterna perché concentra in sé tutte le responsabilità trasversali che altrimenti ogni microservizio dovrebbe gestire autonomamente:

1. Sicurezza e autenticazione:

- tutte le richieste in ingresso vengono validate dal Gateway prima di essere inoltrate ai microservizi interni.
- Un microservizio non autenticato o una richiesta malformata viene bloccata qui, senza mai raggiungere la rete interna.

2. Rate limiting:

- il Gateway limita il numero di richieste che un client può fare in un dato intervallo di tempo, proteggendo il sistema da abusi, attacchi DDoS e sovraccarichi improvvisi.

3. Load balancing:

- distribuisce le richieste in arrivo tra le varie istanze dello stesso microservizio, garantendo che nessuna istanza sia sovraccaricata mentre le altre sono inattive.
- È il meccanismo che rende concreta la scalabilità orizzontale che abbiamo visto in precedenza.

4. Instradamento:

- in base all'URL e al tipo di richiesta, il Gateway determina quale microservizio deve gestirla e la inoltra all'istanza corretta, consultando il Registry per conoscere gli indirizzi aggiornati.

🔗 **Nota il parallelismo con il Controller che abbiamo visto nelle nostre applicazioni Spring:**

esattamente come il **Controller** fa da centralino tra il client e i layer interni dell'applicazione, il Gateway fa da centralino tra internet e la rete interna dei microservizi.

🔔 La differenza è che il Gateway opera a livello infrastrutturale — non contiene logica di business, solo logica di instradamento e protezione.

I Tre Microservizi Infrastrutturali

Ora che abbiamo compreso cos'è il Boundary — il perimetro che contiene tutta la nostra applicazione — possiamo approfondire uno degli aspetti che ho solo accennato: **i servizi infrastrutturali**.

Nel Boundary, accanto ai microservizi di business (come `Catalog`, `Ordering` o `Shopping Cart`), troviamo **tre microservizi speciali** che non contengono logica di business ma sono **indispensabili per il funzionamento stesso dell'architettura**. Sono loro a rendere possibile la coordinazione di un sistema distribuito composto da decine o centinaia di servizi.

Questi tre servizi infrastrutturali sono:

1. Configurazione Centralizzata

Immagina di avere 20 microservizi, ognuno con il proprio file `application.properties` che contiene:

- **l'URL del database,**
- **le credenziali di accesso,**
- **i timeout delle connessioni,**
- **gli endpoint dei servizi esterni.**

Se devi cambiare una configurazione — ad esempio l'URL del database perché stai passando dall'ambiente di sviluppo a quello di produzione — **dovresti modificare 20 file diversi, fare 20 build e riavviare 20 microservizi**.

Un'operazione che da pochi minuti diventerebbe ore.

Il **Config Server** risolve questo problema:

- **centralizzando tutte le configurazioni di tutti i microservizi in un unico luogo.**
- **Ogni microservizio, all'avvio, non legge il proprio file locale ma chiede al Config Server quali sono le sue configurazioni**

- In questo modo le configurazioni diventano così un punto unico di verità, gestito centralmente.

☰ È come se ogni inquilino di un condominio, invece di tenere le proprie chiavi, le ritirasse ogni giorno dal portiere — così se si decide di cambiare la serratura del cancello, basta dare le nuove chiavi al portiere, non a tutti i residenti.

Approfondimento: Perché la Configurazione Centralizzata è Così Importante

Abbiamo detto che il Config Server centralizza le configurazioni di tutti i microservizi.

Ma perché è così fondamentale? Per capirlo, dobbiamo considerare il contesto in cui opera un'applicazione a microservizi.

Un sistema di questo tipo può avere facilmente **decine di servizi**, sviluppati in momenti diversi, da team diversi, e potenzialmente anche con **linguaggi di programmazione diversi** (Java, Python, Node.js...).

Ogni servizio ha le sue configurazioni :

database, credenziali, endpoint esterni, timeout, feature flag, e così via.

Se ogni servizio gestisse la propria configurazione in un file

`application.yml` locale — come abbiamo fatto finora negli esercizi — **ci troveremmo di fronte a tre problemi concreti che emergono in qualsiasi sistema distribuito di medie-grandi dimensioni:**

1. Gestione incoerente:

- ogni team potrebbe organizzare le configurazioni in modo diverso, rendendo difficile capire dove cercare quando qualcosa non funziona.

2. Mancanza di uniformità:

- se un parametro come il timeout delle chiamate HTTP deve essere lo stesso per tutti i servizi, non c'è modo di garantirne l'uniformità se ognuno lo definisce per conto proprio.

3. Modifiche globali impossibili:

- se devi cambiare l'URL del database di produzione per tutti i servizi, dovresti modificare decine di file, fare altrettante build, e

riavviare altrettanti servizi. Un'operazione che da pochi minuti diventerebbe ore o giorni.

Il **Configuration Server** risolve tutti questi problemi grazie a 4 caratteristiche fondamentali:

1. **Separa codice e configurazione:**

- La configurazione non è più impacchettata dentro il JAR del microservizio.
- Diventa un asset esterno, gestito indipendentemente dal codice.
- Questo significa che puoi cambiare un parametro senza dover ricompilare e ridistribuire il servizio.

2. **Centralizza la gestione:**

- Tutte le configurazioni di tutti i microservizi vivono in un unico posto (tipicamente un repository Git).
- Vuoi sapere come è configurato il servizio `Ordering` in produzione? Vai in un posto solo e lo scopri.

3. **Versiona la configurazione:**

- Poiché le configurazioni sono gestite su Git, ottieni automaticamente la cronologia: chi ha cambiato cosa, quando, e perché.
- Puoi fare rollback se qualcosa va storto, esattamente come fai con il codice.

4. **Modifiche senza rebuild:**

- Questa è la vera killer feature:
 - puoi cambiare una configurazione in produzione — ad esempio disabilitare una funzionalità problematica tramite feature flag — e far sì che i microservizi la ricevano senza essere riavviati. Basta un aggiornamento dinamico, nessuna downtime, nessuna ricompilazione.

In sintesi:

Con le configurazioni locali, ogni modifica richiede una nuova build, un nuovo deploy, un riavvio.

Con il Config Server, la configurazione diventa **viva e modificabile** mentre il sistema è in esecuzione.

In un ambiente dove la continuità di servizio è critica, questa non è una comodità — è una necessità.

2. Registry

Abbiamo detto che il Gateway deve inoltrare le richieste ai microservizi corretti. Ma come fa a sapere **dove si trovano** quei microservizi in un dato momento?

In un ambiente dinamico come Kubernetes, i microservizi:

- **si avviano e si spengono continuamente,**
- **cambiano indirizzo IP a ogni riavvio,**
- **vengono scalati orizzontalmente creando nuove istanze.**

Il **Registry** (o Service Discovery) è l'elenco telefonico dell'architettura:

- **Ogni microservizio, quando si avvia, si registra al Registry comunicando: "sono il servizio `Catalog`, mi trovo all'indirizzo IP `10.0.1.23` sulla porta `8081`".**
- **Il Gateway, quando riceve una richiesta per il servizio `Catalog`, chiede al Registry: "dove si trova `Catalog` in questo momento?".**
- **Il Registry risponde con l'indirizzo aggiornato, e il Gateway inoltra la richiesta.**

Se un'istanza muore o viene sostituita, il Registry ne viene a conoscenza e aggiorna la propria lista.

Il Gateway non ha mai bisogno di conoscere indirizzi statici — chiede sempre al Registry l'indirizzo corretto in quel momento.

3. Gateway

Del Gateway abbiamo già parlato in dettaglio nella sezione precedente — è il punto di ingresso unico che fa da "portiere" dell'intera architettura. Vale la pena però richiamare le sue responsabilità alla luce dei due servizi appena introdotti, perché ora ha senso capire **perché** fa quello che fa:

- **Riceve la richiesta dal client esterno — è l'unico punto di contatto tra internet e la rete interna**
- **Autentica e valida la richiesta — applica le regole di sicurezza prima che la richiesta raggiunga qualsiasi microservizio**

- Applica rate limiting — protegge il sistema da abusi, attacchi e sovraccarichi
- Chiede al Registry dove si trova il microservizio destinatario — non usa indirizzi statici, interroga il Registry per ottenere l'indirizzo aggiornato in quel momento
- Inoltra la richiesta all'istanza corretta — eventualmente bilanciando il carico se ci sono più istanze attive
- Restituisce la risposta al client

Questo ci porta a capire la **trilogia infrastrutturale** nel suo insieme:

1. Il **Config Server**:
 - dice a ogni microservizio come deve configurarsi
2. Il **Registry**:
 - dice al Gateway dove trovare i microservizi
3. Il **Gateway**:
 - è il punto di ingresso che usa queste informazioni per instradare le richieste

🕒 **Senza uno solo di questi tre componenti, un'applicazione a microservizi di medie-grandi dimensioni diventerebbe impossibile da gestire.**

1. **Il Config Server è il servizio zero** — parte per primo.
2. **Il Registry parte subito dopo**.
3. **Il Gateway parte per ultimo** — solo quando entrambi sono già attivi e pronti a rispondere.

Spring Cloud Config — L'implementazione di riferimento

Finora abbiamo parlato del Config Server in termini concettuali — **un componente che centralizza le configurazioni**. Ma come si realizza concretamente in un ecosistema Spring?

Il framework che implementa questo pattern si chiama **Spring Cloud Config**:

- **È una soluzione standardizzata per fornire una configurazione centralizzata in applicazioni distribuite, e in particolare in architetture a microservizi basate su Spring Boot**.

Il sistema si compone di due attori:

1. **Il Configuration Server** — un microservizio speciale il cui unico compito è:
 - Recuperare le configurazioni da un backend di storage (un repository Git, un database, o anche il semplice file system)
 - Esporre queste configurazioni via **API HTTP** ai microservizi che le richiedono
2. **Il Configuration Client** — tutti gli altri microservizi dell'applicazione:
 - All'avvio, invece di leggere il proprio `application.yml` locale, fanno una chiamata HTTP al Configuration Server dicendo: "dammi la configurazione per il servizio `Catalog` nell'ambiente `production`"
 - Ricevono la risposta e si configurano di conseguenza

Un dettaglio cruciale: l'ordine di avvio

C'è un aspetto FONDAMENTALE da tenere a mente:

 **Il Configuration Server è il primo servizio che va avviato in tutta l'architettura.**

Ha senso, se ci pensi:

- tutti gli altri microservizi, per potersi configurare, hanno bisogno che il Config Server sia già attivo e in ascolto.
- Se il Config Server non è partito, nessun altro microservizio può avviarsi correttamente.

È il servizio zero — quello da cui tutto dipende, e che quindi deve essere il più stabile e resiliente possibile.

Esempio concreto

Vediamo come si configura concretamente un Configuration Server tramite il suo `application.properties`:



`.properties`

```
1 # Identità del servizio
2 spring.application.name = config-server
3
4 # Profilo attivo
5 spring.profiles.active = native
```

```

6
7 # Configurazione per il profilo "native" - configurazioni lette
  dal file system
8 spring.cloud.config.server.native.search-locations =
  classpath:/config
9
10 # Disabilita compatibility verifier
11 spring.cloud.compatibility-verifier.enabled = false
12
13 # Porta su cui il Config Server rimane in ascolto
14 server.port = 8071

```

Vale la pena capire cosa fa ogni proprietà:

1. `spring.application.name = config-server` :
 - **Come per ogni microservizio Spring Boot, diamo un nome a questo servizio. Nel caso del Configuration Server,**
 - **la convenzione è chiamarlo `config-server`.**
 - **Questo nome sarà utilizzato dal Registry per identificarlo.**
2. `spring.profiles.active = native` :
 - Spring Cloud Config supporta diversi backend di storage per le configurazioni (chiamati "profili"):
 - `native` :
 - **le configurazioni vengono lette dal file system locale,**
 - Utile in fase di sviluppo
 - `git` :
 - **==le configurazioni vengono lette da un repository Git ==**
 - la soluzione più comune in fase di produzione
 - `vault` :
 - **le configurazioni vengono lette da HashiCorp Vault (per secret management)**

In questo esempio usiamo `native` per semplicità — utile in sviluppo o per capire il meccanismo.

3. `spring.cloud.config.server.native.search-locations = classpath:/config` :
 - **Con il profilo `native`, questa proprietà dice al server dove trovare i file di configurazione. `classpath:/config` significa "cerca nella cartella `config/` all'interno del JAR".**

- In alternativa si può usare un percorso assoluto sul filesystem, ad esempio `file:/etc/config-server/`.
4. `spring.cloud.compatibility-verifier.enabled = false` :
- **Disabilita il controllo di compatibilità tra le versioni di Spring Boot e Spring Cloud.**
 - **In alcuni ambienti questo controllo può generare warning fastidiosi quando le versioni non sono perfettamente allineate.**
5. `server.port = 8071` :
- **La porta su cui il Configuration Server espone le sue API HTTP.**
 - **I microservizi client faranno chiamate a questa porta per recuperare la loro configurazione.**
 - 8071 è un valore di esempio — in produzione potrebbe essere un'altra porta, ma è importante che sia **fissa e ben nota** dato che tutti i servizi devono sapere dove trovarlo.

I due approcci principali:

- `native` — **semplice, ideale per sviluppo e test, ma richiede di distribuire i file di configurazione insieme al server**
- `git` — l'approccio standard in produzione:
 - **le configurazioni vivono in un repository Git, ottenendo versionamento, audit trail e aggiornamenti senza ricostruire il server**

Nelle prossime sezioni vedremo come si configura un microservizio client per interrogare questo server all'avvio e recuperare le proprie impostazioni.

Il Registry — Il DNS dei Microservizi

Abbiamo visto che il Configuration Server è il primo servizio da avviare — il "servizio zero".

Subito dopo, nel boot order dell'architettura, c'è il secondo pilastro infrastrutturale: il **Registry** (chiamato anche Service Discovery).

Ma cosa fa esattamente il Registry? Il suo scopo è semplice da capire ma cruciale per il funzionamento:

- **gestisce e mantiene aggiornata la lista di tutti i microservizi disponibili, con tutti i dettagli necessari per contattarli: IP, porta, nome logico,**

numero di istanze attive, e stato di salute.

Come funziona nel dettaglio

Il meccanismo è simile a quello di un condominio con un tabellone all'ingresso:

1. **Registrazione all'avvio** — ogni microservizio, quando si avvia, si presenta al Registry e comunica:
 - **il suo nome logico (ad esempio `catalog-service` o `ordering-service` — un nome stabile che non cambia mai)**
 - **il suo indirizzo IP (che in un ambiente dinamico come Kubernetes può cambiare a ogni riavvio)**
 - **la porta su cui è in ascolto**
2. **Heartbeat — il battito cardiaco:**
 - i microservizi non si registrano una volta sola e poi spariscono.
 - Avvisano **continuamente** il Registry del loro stato di vita e del numero di istanze disponibili tramite una comunicazione periodica chiamata **heartbeat**.
 - Grazie a questo meccanismo il Registry sa costantemente chi è in vita e chi no — se un microservizio smette di inviare heartbeat, il Registry lo considera inattivo e lo rimuove dalla lista.
3. **Scoperta per interrogazione** — quando il Gateway (o un altro microservizio) ha bisogno di contattare un servizio, non usa indirizzi fissi. Invece:
 - **chiede al Registry: "dove si trova `catalog-service` in questo momento?"**
 - **il Registry risponde con l'indirizzo IP e la porta dell'istanza (o delle istanze) attualmente disponibili**

☰ Analogia:

Il Registry è il **DNS** dei microservizi:

Come per il DNS, che trasforma un nome di dominio (come

`www.google.com`) in un **indirizzo IP**, il Registry trasforma un nome logico di servizio (come `ordering-service`) nell'**indirizzo IP** e porta dove quel servizio è effettivamente in esecuzione in quel momento.

La differenza è che il Registry è dinamico:

- gli indirizzi cambiano in tempo reale mentre i servizi si avviano, si spengono e si scalano.



Eureka — Il Service Registry di Netflix

L'implementazione più famosa e utilizzata di questo pattern è **Eureka**, un Service Registry open source sviluppato da Netflix e integrato in Spring Cloud.

Cosa fa Eureka:

Eureka è un microservizio speciale che svolge tre compiti fondamentali:

1. **Permette ai servizi di registrarsi all'avvio** — ogni microservizio client contiene un "Eureka Client" che, quando parte, chiama Eureka Server dicendo "sono qui".
2. **Permette ai servizi di scoprire dinamicamente gli altri** — quando un servizio (o il Gateway) ha bisogno di chiamare un altro servizio, chiede a Eureka: "dove si trova? quali istanze sono attive?".
3. **Espone il registro via API REST** — tutto ciò che Eureka sa è accessibile tramite semplici chiamate HTTP. Puoi puntare un browser all'endpoint di Eureka e vedere in tempo reale quali servizi sono registrati.
4. **Mantiene numero e stato delle istanze** — Eureka sa non solo quali servizi esistono, ma anche quante istanze di ciascuno sono attive e se sono "UP" (salute) o "DOWN" (guaste).

Un chiarimento importante:

Il Registry **non si occupa dell'instradamento** — non decide dove mandare le richieste, non fa load balancing, non inoltra nulla.

Il Registry si limita a fornire informazioni su dove si trovano i servizi.

L'instradamento è responsabilità del Gateway (o del client che fa la chiamata).

L'ordine di avvio: Registry dopo Config

Abbiamo detto che il Configuration Server è il primo servizio ad avviarsi. Eureka è il **secondo**. C'è una ragione precisa per questo ordine:

Il Registry DEVE caricare la sua configurazione dal Config Server prima di potersi avviare.

Questo significa che:

1. **Prima** si avvia il Config Server (porta 8071 nell'esempio precedente)
2. **Poi** si avvia Eureka, che:
 - Interroga il Config Server per sapere:
 - **su quale porta deve ascoltare**
 - **qual è il proprio nome**
 - **dove trovare eventuali backend di persistenza**
 - **parametri di comportamento (tempi di heartbeat, soglie di eliminazione, ecc.)**
 - Dopo aver ricevuto la configurazione, si avvia e si mette in ascolto per le registrazioni

Esempio di Configurazione con Eureka

La configurazione di Eureka si articola in due file: uno per il **Registry Server** e uno per il **Config Server** che gli fornisce le impostazioni.

Properties file del Registry Server (Eureka):



.properties

```
1  # Nome del servizio
2  spring.application.name = MM_quiz_Eureka_Server
3  # Profilo attivo
4  spring.profiles.active = dev
5  # URL del Config Server da cui recuperare la configurazione
6  spring.config.import =
    >optional:configserver:http://localhost:8071
```

La riga più importante è `spring.config.import` — **dice a Eureka di recuperare la propria configurazione dal Config Server attivo sulla porta 8071**.

È la conferma pratica dell'ordine di avvio:

- Eureka sa dove trovare il Config Server perché quell'indirizzo è l'unica cosa configurata localmente.

Properties file nel Config Server (configurazione di Eureka):



.properties

```
1 # Proprietà di esempio
2 example.property = Eureka server
3 # Porta su cui Eureka rimane in ascolto
4 server.port = 8070
5 # Hostname dell'istanza
6 eureka.instance.hostname = localhost
7 # Eureka non si registra su se stesso
8 eureka.client.registerWithEureka = false
```

Vale la pena soffermarsi su `eureka.client.registerWithEureka = false` — questa proprietà dice a Eureka di non registrare se stesso nel proprio registry.

Ha senso: Eureka è il registry, non un microservizio che ha bisogno di essere scoperto come gli altri.

La Dashboard di Eureka

Per monitorare lo stato del registry basta aprire il browser e navigare all'indirizzo:



.properties

```
1 http://eurekaIP:eurekaPort
```

Nel nostro esempio:



.properties

```
1 http://localhost:8070
```

La dashboard mostra in tempo reale tutte le istanze registrate, il loro stato (UP o DOWN) e quante istanze di ogni servizio sono attive.

Ad esempio se hai scalato orizzontalmente il servizio `MM-QUIZ-RD`, la dashboard mostrerà **2 istanze UP** per quel servizio — confermando che il load balancing è attivo e funzionante.

❓ Cosa succede se Eureka non è attivo?

Le conseguenze sono a cascata e bloccano l'intera architettura:

- **I microservizi non si registrano** — all'avvio, ogni microservizio cerca Eureka per registrarsi. Se Eureka non risponde, il microservizio può:
 - **andare in errore e terminare**
 - **continuare a ritentare in loop, ma rimanere "invisibile" agli altri**
- **Il Gateway non riesce a fare service discovery** — **il Gateway, quando riceve una richiesta per, ad esempio, `catalog-service`, chiede a Eureka: "dove si trova?" Senza Eureka, non ottiene risposta, e la richiesta del client fallisce.**

In pratica, **senza Eureka, l'intero sistema è paralizzato.**

Per questo motivo, nella sequenza di boot di un'applicazione a microservizi, Eureka è tra i primi componenti ad essere avviati — subito dopo il Config Server, ma prima di qualsiasi microservizio applicativo.

Il Gateway — Nel Dettaglio

Abbiamo già introdotto il [Gateway](#) come punto di ingresso unico dell'architettura. Ora vediamo nel dettaglio come funziona il suo meccanismo di instradamento.

Come Instrada le Richieste

Quando un client — o anche un microservizio interno — vuole comunicare con un altro microservizio, non usa l'indirizzo IP diretto.

Usa il **nome logico** del servizio destinatario, esattamente come funziona il DNS:



HTTP

```
1 http://gatewayIP:8073/serviceCatalog/param...
```

A questo punto il Gateway esegue tre operazioni in sequenza:

1. Si interfaccia con il [Registry](#):

- **comunicando il nome logico del servizio richiesto** — nel nostro **esempio** `serviceCatalog`

2. **Ottiene l'URL effettiva dell'istanza attiva in quel momento dal Registry**
3. **Procede all'instradamento:**
 - **inoltrando la richiesta all'indirizzo reale del microservizio:**



HTTP

```
1 http://serviceIP:servicePort/param...
```

Il client non sa e non gli importa dove si trova fisicamente il microservizio — **sa solo il nome logico.**

È il Gateway a fare da traduttore tra il nome logico e l'indirizzo reale, consultando il Registry ad ogni richiesta.

❓ **Perché è l'ultimo servizio ad avviarsi?**

Perché per funzionare correttamente deve trovare il Config Server già attivo:

- **per recuperare la sua configurazione, e il Registry già attivo — per poter risolvere i nomi logici dei servizi.**

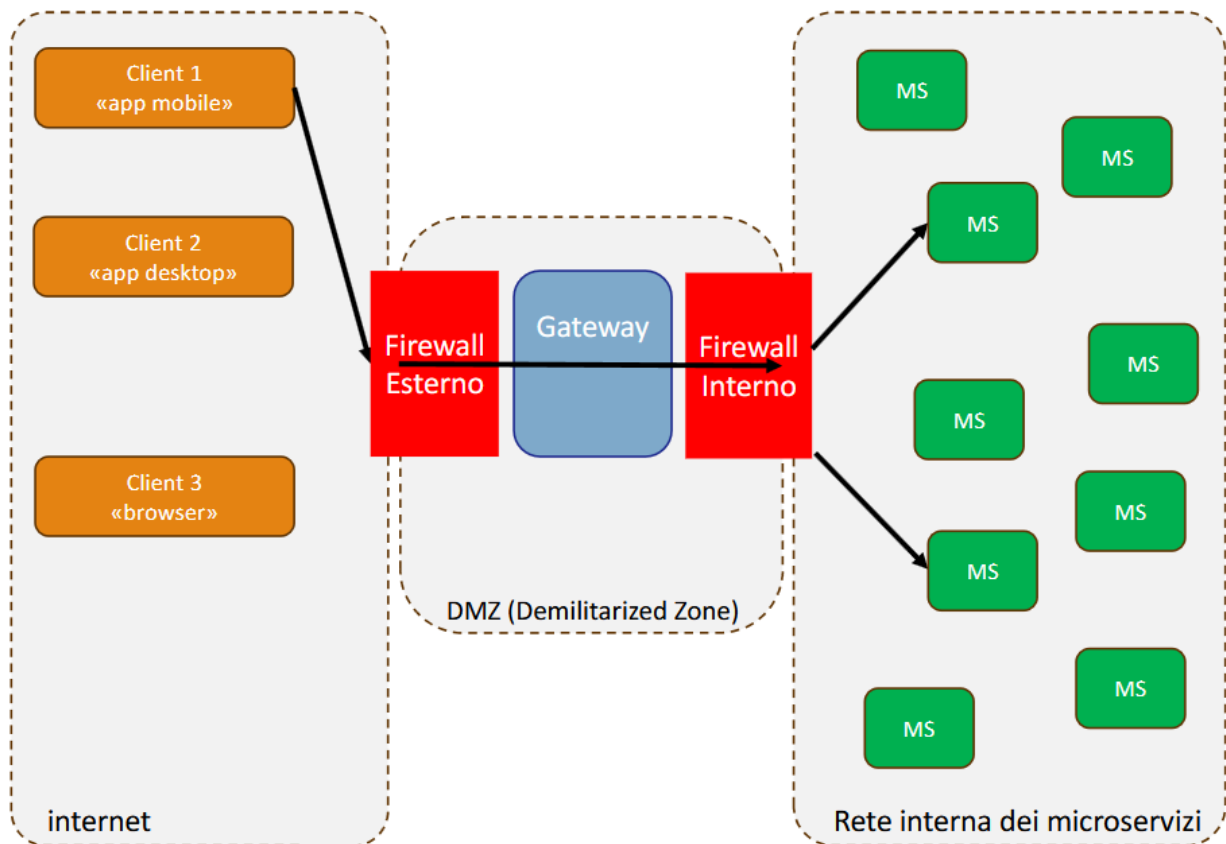
Se partisse prima di uno dei due, non riuscirebbe né a configurarsi né a fare service discovery.

La DMZ e la Protezione della Rete Interna

Il Gateway non opera da solo — fa parte di un sistema di protezione a più livelli chiamato **DMZ (Demilitarized Zone)**:

- **separa la rete non protetta (internet) dalla rete interna dove vivono i microservizi.**

Per comprendere meglio come Il Gateway entra a far parte di questo sistema di protezione a più livelli analizziamo questa immagine:



Partendo da sinistra abbiamo 3 client:

1. **Client 1:** Un applicazione mobile.
2. **Client 2:** Un applicazione desktop.
3. **Client 3:** Un Browser web.

Il **Client 1** invia una richiesta http alla rete dei microservizi, come abbiamo già visto in precedenza però, il o i client non comunicano direttamente con la rete ma devo passare per il gateway che in base alle richieste ricevute indirizza il o i client ai corretti microservizi richiesti.

Ed è proprio qui che entra in gioco la DMZ.

Il **Client 1** invia la richiesta e questa richiesta passa per il **Firewall Esterno**:

- ==primo filtro che blocca le chiamate indesiderate provenienti da internet. ==
- **La rete esterna conosce solo l'IP del firewall — non sa nulla di quello che c'è nella rete interna.**
- **È la prima barriera di difesa.**

Una volta che questo firewall ha "filtrato" le richieste indesiderate(es: malware in ingresso, etc.) la richiesta entra nella **DMZ vera e propria** — la zona franca tra internet e la rete interna, in cui potenzialmente potrebbe essere messa sotto attacco (es: man in the middle), ed è proprio qui che opera l'**API Gateway**:

- **unico punto di accesso autorizzato alla rete interna.**
- **Tutte le richieste che superano il firewall esterno passano obbligatoriamente attraverso il Gateway, che le autentica, le valida e le instrada ai microservizi corretti.**

Una volta che la richiesta passa l'autenticazione e la validazione del Gateway incontra un ulteriore livello, chiamato **Firewall Interno** :

- **secondo livello di protezione nel caso in cui il firewall esterno venga superato.**
- **È una rete di sicurezza aggiuntiva che protegge i microservizi anche da attacchi che riuscissero a bypassare il primo filtro.**

Grazie a questa struttura a tre livelli:

- **i microservizi nella rete interna possono comunicare tra loro in un ambiente sicuro — senza doversi preoccupare di filtrare o validare le richieste in ingresso, perché ci ha già pensato la DMZ.**

Gateway — Responsabilità Aggiuntive e Pattern di Aggregazione

Abbiamo visto che il Gateway è il punto di ingresso unico dell'architettura — **la porta d'accesso tra internet e la rete interna.**

Essendo l'unico componente esposto verso il mondo esterno, **può (e dovrebbe) accentrare in sé anche altre responsabilità comuni a tutti i microservizi.**

Le Responsabilità Trasversali del Gateway

Oltre all'instradamento di base, il Gateway è il posto ideale per gestire:

- **Sicurezza di base — autenticazione delle richieste in ingresso, validazione dei token JWT, gestione delle sessioni**
- **Aggregazione di dati — combinare risposte provenienti da più microservizi in un unico risultato, riducendo il numero di chiamate che il client deve effettuare**
- **ID di correlazione — generare un identificatore unico per ogni richiesta e propagarlo a tutti i microservizi coinvolti, permettendo di tracciare il percorso di una chiamata attraverso l'intero sistema**
- **Cambio del formato dei dati — trasformare il formato di risposta (es. da XML a JSON) in base al client che ha fatto la richiesta**

- **Rate limiting** — limitare il numero di richieste per client per prevenire abusi
- **Caching** — memorizzare risposte frequenti per ridurre il carico sui microservizi interni
- **Logging centralizzato** — registrare tutte le richieste in ingresso in un unico posto

👁 Il principio alla base:

Tutto ciò che è trasversale (riguarda più microservizi) e non è logica di business dovrebbe stare nel Gateway.

Questo permette a ogni singolo microservizio di concentrarsi esclusivamente sulla propria logica di dominio, senza dover gestire preoccupazioni infrastrutturali.

Esempio Concreto: L'Aggregazione di Dati

Per capire l'importanza dell'aggregazione, consideriamo un esempio pratico.

Immagina un'applicazione e-commerce che deve mostrare una pagina prodotto con:

- i dati del prodotto (nome, prezzo, descrizione)
- le recensioni degli utenti

Nell'architettura a microservizi, questi dati risiedono su due servizi diversi:

- `catalog-rd` — **gestisce i dati anagrafici dei prodotti.** ^{^catalog-rd}
- `feedback-product` — **gestisce le recensioni.** ^{^feedback-product}

Senza un Gateway intelligente, il client (browser o app mobile) dovrebbe:

1. Chiamare `catalog-rd` per ottenere i dati del prodotto
2. Chiamare `feedback-product` per ottenere le recensioni
3. Attendere entrambe le risposte
4. Aggregare i dati e renderizzarli

Questo approccio ha diversi problemi:

- **Il client diventa complesso, dovendo gestire chiamate multiple e aggregazione**

- **Aumenta la latenza (due chiamate invece di una)**
- **Consumo maggiore di banda sulla rete esterna**
- **Se uno dei due servizi cambia API, devi aggiornare il client**

Con un Gateway che fa aggregazione, il flusso cambia radicalmente:



HTTP

```
1 GET /product-summary/{id}
```

Il Gateway espone un unico endpoint `/product-summary/{id}`.

Quando riceve la richiesta:

1. **Chiama** `catalog-rd` **per ottenere i dati del prodotto**
2. **Chiama** `feedback-product` **per ottenere le recensioni**
3. **Aggrega i dati in un'unica risposta strutturata**
4. **Restituisce tutto al client in una sola chiamata**

Il client fa **una sola richiesta** e riceve **tutti i dati** già aggregati.

Vantaggi di questo approccio:

- Il client è più semplice — chiama un endpoint solo
- Minore latenza percepita — una chiamata invece di due
- Se cambia l'API di `catalog-rd`, si modifica solo il Gateway, non i client
- Il Gateway può ottimizzare le chiamate interne (es. eseguendole in parallelo)



Il Gateway come Microservizio a Sé Stante

È importante notare un dettaglio fondamentale:

Il Gateway non è un componente magico o un software pre-confezionato. È un microservizio a tutti gli effetti, scritto e mantenuto dal team di sviluppo.

Il Gateway:

- **Ha il suo codice**

- Ha la sua logica di aggregazione e instradamento
- Si registra al Registry come qualsiasi altro servizio
- Può essere scalato orizzontalmente se necessario
- Segue lo stesso ciclo di sviluppo, test e deploy degli altri microservizi

L'unica differenza è che non contiene logica di business — contiene solo logica di infrastruttura e orchestrazione.

Ma dal punto di vista tecnico, è un microservizio come tutti gli altri.

❓ Perché il Gateway è l'ultimo servizio ad avviarsi?

Come abbiamo visto nella sezione precedente, il Gateway dipende da:

- Config Server — per recuperare la sua configurazione (su quale porta ascoltare, quali endpoint esporre, ecc.)
- Registry — per risolvere i nomi logici dei servizi e scoprire dove si trovano le istanze attive

Per questo motivo, nella sequenza di boot dell'architettura:

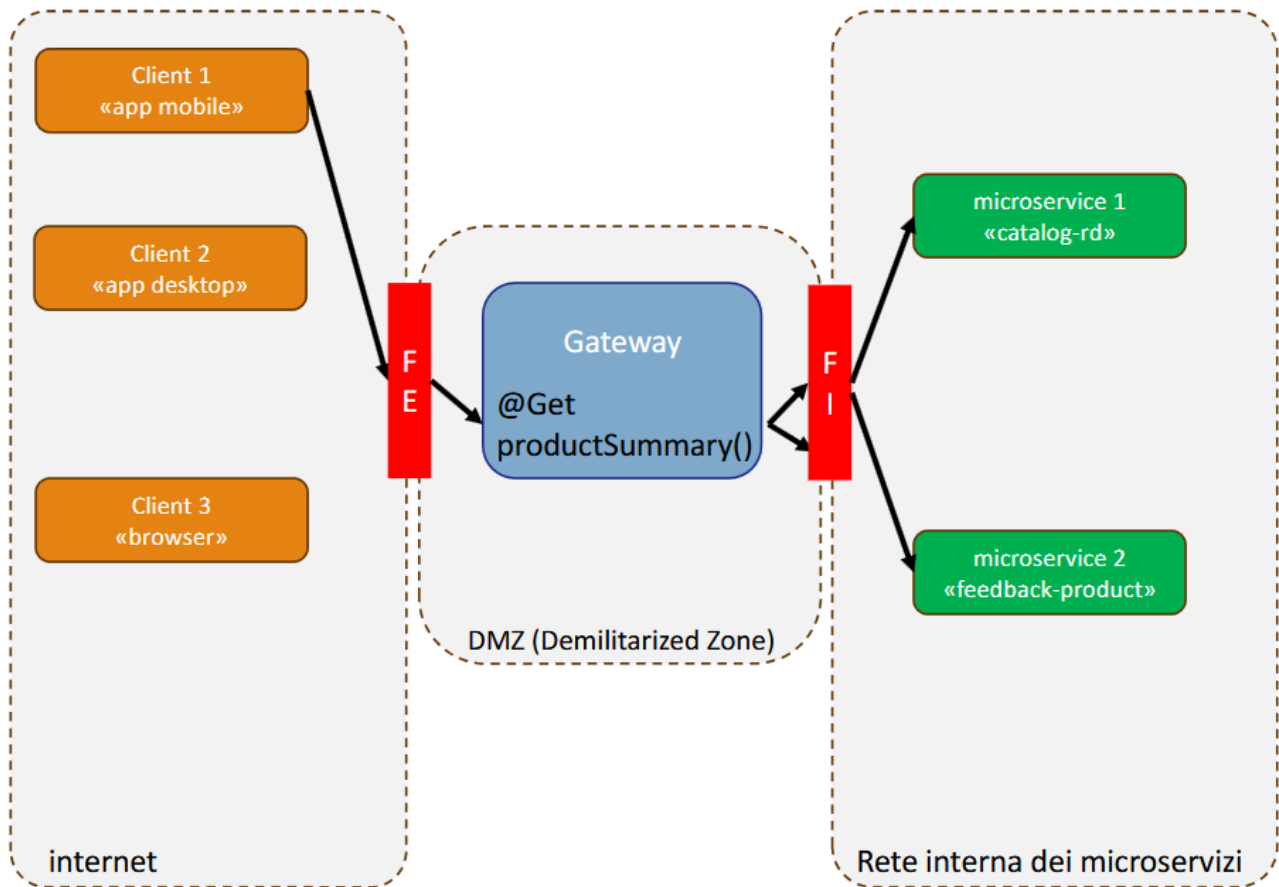
1. Config Server
2. Registry (Eureka)
3. Gateway
4. Microservizi applicativi

Se il Gateway partisse prima del Config Server o del Registry, non potrebbe né configurarsi né fare service discovery — e quindi non sarebbe in grado di gestire alcuna richiesta.

Il Gateway come Servizio Infrastrutturale — Un Vincolo Fondamentale

L'esempio di aggregazione che abbiamo appena visto — con il Gateway che combina dati da `catalog-rd` e `feedback-product` — solleva una domanda cruciale: **dove finisce l'aggregazione "infrastrutturale" e dove inizia la logica di business?**

Guardiamo l'immagine:



Nello schema vediamo:

- **Tre client** (mobile, desktop, browser) che puntano al Gateway nella DMZ
- **Il Gateway** che espone un endpoint `@Get productSummary()`
- **Due microservizi** nella rete interna: `catalog-rd` e `feedback-product`

Il Vincolo Fondamentale

🔔 **L'aggregazione nel Gateway ha senso SOLO se il servizio presente sul Gateway non aggiunge logica di business.**

Cosa significa esattamente?

Il Gateway deve limitarsi a:

- **Chiamare i microservizi necessari**
- **Attendere le loro risposte**
- **Combinare i dati ricevuti in un'unica struttura**
- **Restituire il risultato al client**

Il Gateway NON deve:

- **Applicare regole di business** (es. "se il prezzo è inferiore a 10€, applica uno sconto")
- **Validare dati secondo logiche di dominio** (es. "il prodotto non può essere ordinato se è esaurito")
- **Calcolare valori che dipendono da regole aziendali** (es. "calcola il totale dell'ordine con le tasse")
- **Modificare i dati in modo che alteri il loro significato di business**

Perché Questo Vincolo è Così Importante?

Il Gateway è un **servizio infrastrutturale**, non un microservizio di business. Questa distinzione ha implicazioni profonde:

1. Separazione delle responsabilità

- **La logica di business vive solo nei microservizi applicativi** (`catalog-rd`, `feedback-product`, `ordering`, ecc.)
- **Il Gateway si occupa solo di orchestrazione, sicurezza e instradamento**
- **Se la logica di business finisse nel Gateway, creeremmo accoppiamento e duplicazione**

2. Manutenibilità

- **Le regole di business cambiano spesso — se sono sparse tra Gateway e microservizi, modificarle diventa un incubo**
- **Con la logica di business confinata nei microservizi, si cambia un solo posto**

3. Riutilizzabilità

- **Il Gateway espone un endpoint aggregato** `productSummary()` **per i client esterni**
- **Ma se un microservizio interno avesse bisogno di fare la stessa aggregazione, chiamerebbe direttamente** `catalog-rd` **e** `feedback-product`, **non passerebbe dal Gateway**
- **Se la logica di business fosse nel Gateway, i microservizi non potrebbero riutilizzarla**

4. Testabilità

- **Il Gateway, senza logica di business, è più semplice da testare — basta verificare che chiami i servizi giusti e combini correttamente le risposte**
- **I test di business rimangono nei microservizi dove devono stare**

☰ Un'Analogia per Capire:

Pensa al Gateway come a un **centralinista** di un grande ufficio:

- Il centralinista riceve una chiamata: "Mi servono le informazioni del dipartimento Vendite e del dipartimento Supporto sul cliente X"
- Il centralinista chiama il dipartimento Vendite e il dipartimento Supporto, aspetta le risposte, le mette insieme, e le riferisce al chiamante

Quello che il centralinista **non** fa:

- Non decide se il cliente ha diritto a uno sconto (lo decide Vendite)
- Non stabilisce se la pratica è prioritaria (lo decide Supporto)
- Non modifica le informazioni che riceve

Il centralinista **orchestra** la comunicazione, ma **non aggiunge contenuto** alle informazioni che trasmette.

Nel Nostro Esempio

Con `productSummary()`, il Gateway:

- ✓ **Chiama** `catalog-rd` **e** `feedback-product`
- ✓ **Attende le risposte di entrambi**
- ✓ **Combina i dati in un JSON unico**
- ✓ **Restituisce il risultato**

✗ Cosa non deve fare:

- ✗ Applicare filtri tipo "mostra solo recensioni con valutazione > **3 stelle**" — **sarebbe logica di business di** `feedback-product`
- ✗ Calcolare il prezzo con sconto in base al profilo utente — **sarebbe logica di business di** `catalog-rd`
- ✗ Decidere di non mostrare un prodotto se esaurito — **sarebbe logica di business di** `catalog-rd`

📄 In sintesi:

Il Gateway è un **servizio infrastrutturale**, non un microservizio di business.

Può aggregare dati da più servizi, ma non deve aggiungere logica di business.

Se hai bisogno di logica di business nell'aggregazione, significa che dovresti creare un nuovo microservizio applicativo dedicato a quella funzionalità, e lasciare il Gateway al suo ruolo di puro orchestratore infrastrutturale.

Quando l'Aggregazione Richiede Logica di Business — Il Microservizio Orchestratore

Abbiamo appena stabilito un principio fondamentale:

- **il Gateway non deve contenere logica di business.**

Ma cosa succede quando l'aggregazione di dati richiede proprio quella logica?

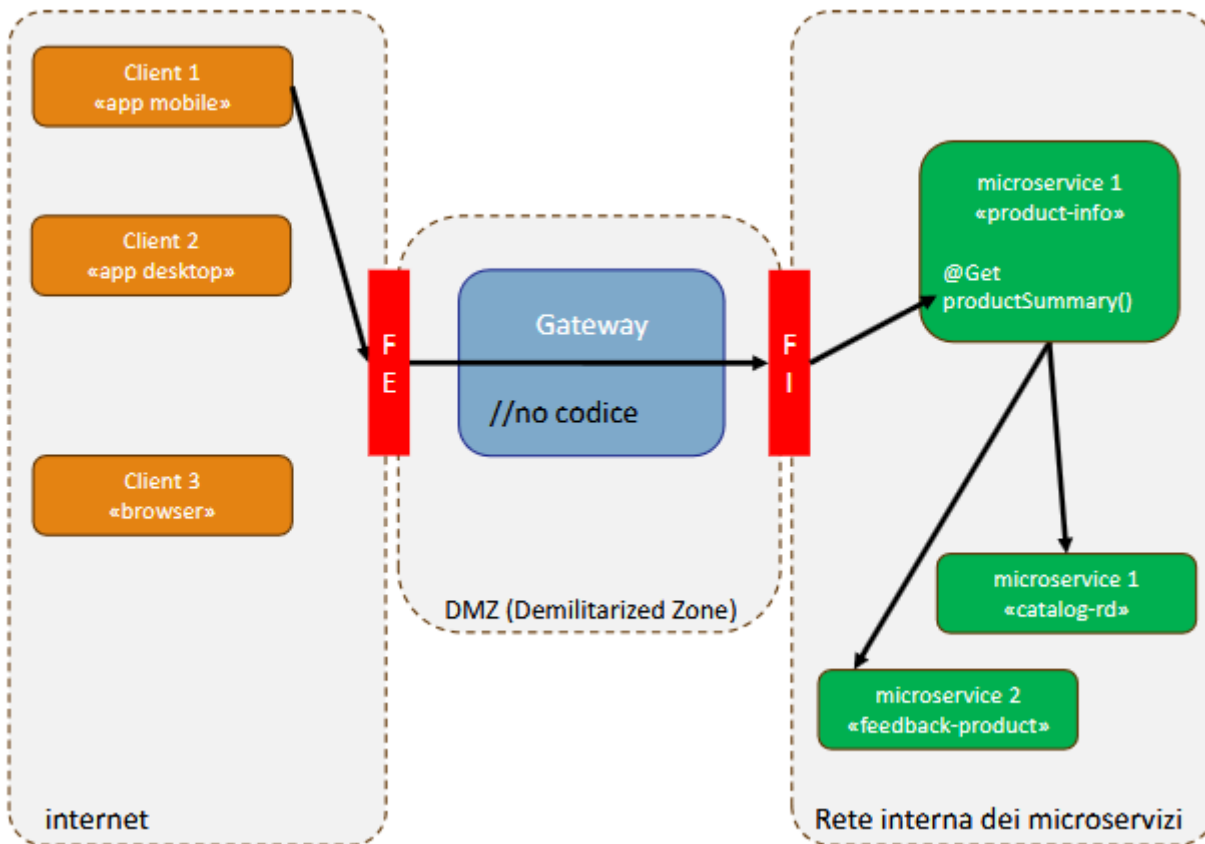
Ad esempio:

- "Se l'utente è Premium, applica uno sconto del 10% sul prezzo del prodotto"
- "Se il prodotto è esaurito, cerca un'alternativa simile nel catalogo"
- "Se la recensione media è inferiore a 3 stelle, mostra un avviso"

In questi casi, **non possiamo mettere questa logica nel Gateway** — violeremmo il principio di separazione.

La soluzione è creare un microservizio dedicato che si occupa di orchestrazione con logica di business.

Guardiamo l'immagine:



La Soluzione: Un Microservizio Dedicato

Nello schema vediamo una struttura a tre livelli nella rete interna:

1. Il Gateway:

- rimane "pulito" (nel diagramma: F E //no codice), senza logica di business.
- Fa solo da punto di ingresso.

2. Il microservizio orchestratore — product-info :

- Espone l'endpoint @Get productSummary()
- Contiene la logica di business (es. sconto per utenti Premium, gestione esaurimento scorte, ecc.)
- Chiama i servizi sottostanti per ottenere i dati grezzi
- Applica le regole di business
- Restituisce il risultato aggregato e "processato"

3. I microservizi di dominio — catalog-rd e feedback-product :

- Forniscono dati grezzi senza logica di contesto
- Sono riutilizzabili da più orchestratori

✓ **Vantaggi di Questo Approccio**

- **Separazione chiara:**
 - **il Gateway fa solo da proxy/infrastruttura; la logica di business vive in microservizi dedicati**
- **Riusabilità:**
 - **product-info può essere chiamato da più client (Gateway, altri microservizi, batch, ecc.)**
- **Testabilità:**
 - **la logica di business è isolata in un microservizio che può essere testato autonomamente**
- **Scalabilità indipendente:**
 - **se product-info è molto richiesto, può essere scalato senza toccare il Gateway o i servizi di dominio**

Il Gateway — Perché per Ultimo?

Abbiamo già accennato all'ordine di avvio nell'architettura. Ora lo formalizziamo:

Il Gateway è l'ultimo servizio infrastrutturale ad avviarsi.

La ragione è semplice, **il Gateway ha due dipendenze obbligatorie che devono essere già attive quando lui parte:**

1. Il Configuration Server:

- **deve essere attivo perché il Gateway deve recuperare la propria configurazione (porta di ascolto, route da esporre, parametri di sicurezza, ecc.)**

2. Il Registry (Eureka):

- **deve essere attivo perché il Gateway, per creare le route dinamiche, deve poter risolvere i nomi logici dei servizi verso cui inoltrare le richieste**

Cosa Succede se il Gateway Parte Prima del Registry?

Le conseguenze sono immediate:

- **Il Gateway non trova i servizi nel Registry**
- **Non riesce a creare le route dinamiche — non sa a quali indirizzi IP inoltrare le richieste**

- Anche se i microservizi applicativi si avviassero dopo, il Gateway non avrebbe modo di "scoprirli" senza un riavvio

Per questo, nella sequenza corretta:

1. Config Server (porta 8071)
2. Registry — Eureka (porta 8761 di default)
3. **Gateway** (porta 8073, come vedremo)
4. Microservizi applicativi

Esempio di configurazione del gateway — Due File, Due Responsabilità

Nell'architettura basata su Spring Cloud, la configurazione del Gateway è distribuita su due file distinti, ognuno con una responsabilità specifica. Questo riflette il principio di separazione che abbiamo visto: **il Gateway carica la configurazione dal Config Server.**

File 1: `bootstrap.properties` (o `application.properties`) del Gateway

Questo file contiene **ciò che il Gateway deve sapere per avviarsi e trovare il Config Server:**



`.properties`

```
1  # Identità del servizio
2  spring.application.name=MM-quiz-Gateway-Server
3  spring.profiles.active=dev
4
5  # Dove trovare il Configuration Server?
6  spring.config.import=optional\:configserver\:http\://localhost\:80
7
8  # Generazione automatica delle route basata sui servizi registrati
   su Eureka
9  spring.cloud.gateway.server.webflux.discovery.locator.enabled=true
10 spring.cloud.gateway.server.webflux.discovery.locator.lower-case-
    service-id=true
11
12 # Esposizione degli endpoint di gestione per monitoraggio
13 management.endpoint.gateway.access=unrestricted
14 management.endpoints.web.exposure.include=gateway, refresh
```

Analizziamo riga per riga:

1. `spring.application.name=MM-quiz-Gateway-Server` :

- **Il nome logico del Gateway.**
 - **Questo nome verrà registrato su Eureka e potrà essere usato da altri servizi per localizzarlo (anche se raramente serve, dato che è il Gateway a chiamare gli altri, non il contrario).**
2. `spring.profiles.active=dev` :
- **Attiva il profilo "dev". Il Config Server, quando il Gateway gli chiede la configurazione, restituirà le proprietà specifiche per l'ambiente di sviluppo .**
3. `spring.config.import=optional\:configserver\:http\://localhost\:8071` :
- Questa è la riga più importante per l'avvio.
 - Dice al Gateway:
 - `optional`: — **se il Config Server non è raggiungibile, non bloccare l'avvio (utile in sviluppo)**
 - `configserver`: — **stai usando Spring Cloud Config**
 - `http://localhost:8071` — **il Configuration Server si trova a questo indirizzo e porta (coerente con l'esempio della lezione precedente)**
4. `spring.cloud.gateway.server.webflux.discovery.locator.enabled=true` :
- **Abilita la generazione automatica delle route basata sui servizi registrati in Eureka.**
 - **Con questa proprietà, non devi definire manualmente ogni route.**
 - Il Gateway crea automaticamente route del tipo:



HTTP

1 `http://gateway:8073/NOME-SERVIZIO/...`

- dove `NOME-SERVIZIO` è il nome con cui il microservizio si è registrato in Eureka.
5. `spring.cloud.gateway.server.webflux.discovery.locator.lower-case-service-id=true` :
- **Converte i nomi dei servizi in minuscolo nell'URL. Se un servizio si registra come `CATALOG-SERVICE`, la route diventa `http://gateway:8073/catalog-service/...`**
6. `management.endpoint.gateway.access=unrestricted` e `management.endpoints.web.exposure.include=gateway, refresh` :

- Espongono gli endpoint di amministrazione del Gateway via HTTP. Utili per:
- `gateway` — **vedere le route attualmente configurate**
- `refresh` — **forzare il ricaricamento della configurazione senza riavviare**

File 2: La Configurazione sul Config Server

Il secondo file non vive nel Gateway — **è archiviato nel Config Server (su Git, filesystem, ecc.) e viene fornito al Gateway quando questo lo richiede all'avvio**.



`.properties`

```
1 # Configurazione per il Gateway – fornita dal Config Server
2
3 # Specifica la porta del Gateway: 8073
4 example.property = Gateway server
5 server.port = 8073
6
7 # Configurazione Eureka – dove trovare il Registry
8 eureka.client.serviceURL.defaultZone =
  http://localhost:8070/eureka
```

Analizziamo:

1. `server.port = 8073` :
 - **Questa proprietà viene iniettata nel Gateway al momento dell'avvio.**
 - **È il Config Server che dice al Gateway su quale porta deve ascoltare.**
 - **In questo modo, se vuoi cambiare la porta del Gateway, modifichi un solo file (quello sul Config Server) e riavvii il Gateway — senza toccare il suo `bootstrap.properties`.**
2. `eureka.client.serviceURL.defaultZone = http://localhost:8070/eureka` :
 - **Dice al Gateway dove trovare Eureka (il Registry).**
 - In questo esempio, Eureka è in ascolto sulla porta 8070.

Nota: le righe commentate (`eureka.instance.preferIpAddress` , `eureka.client.registerWithEureka` , `eureka.client.fetchRegistry`) sono proprietà comuni per configurare il client Eureka.

Se dis-commentate, permetterebbero al Gateway di registrarsi a sua volta su Eureka.

❓ Perché Questa Separazione?

La separazione in due file non è casuale — riflette l'architettura che abbiamo costruito:

File	Dove vive	Cosa contiene	Perché
<code>bootstrap.properties</code>	Nel Gateway stesso	Indirizzo del Config Server, nome del servizio	Deve essere minimale — solo ciò che serve per trovare il Config Server
Configurazione su Config Server	Nel repository centralizzato	Porta, indirizzo di Eureka, parametri di business	Tutto il resto — può essere modificato senza rebuild del Gateway

In pratica:

1. Il Gateway si avvia con un `bootstrap.properties` **minimale**
2. Chiama il Config Server all'indirizzo specificato
3. Riceve la configurazione completa (porta, Eureka, ecc.)
4. Si configura di conseguenza e si mette in ascolto

✓ Vantaggio:

Se vuoi cambiare la porta del Gateway o l'indirizzo di Eureka, modifichi la configurazione sul Config Server e riavvii il Gateway.
Nessuna modifica al codice, nessuna rebuild, nessun nuovo JAR da distribuire.

Con queste configurazioni, l'ordine di avvio diventa:

1. **Config Server** — deve essere attivo sulla porta 8071
2. **Eureka (Registry)** — deve essere attivo sulla porta 8070
3. **Gateway** — si avvia, carica la configurazione dal Config Server (porta 8071), si registra su Eureka (porta 8070), e si mette in ascolto sulla porta 8073
4. **Microservizi applicativi** — si avviano, si registrano su Eureka, e il Gateway può instradare le richieste verso di loro

fino a pag 44